

LAPORAN PRAKTIKUM STRUKTUR DATA

Stack

DISUSUN OLEH:

Rayya Syaquinah Putri Hasibuan

2511532007

DOSEN PENGAMPU:

Dr. Wahyudi, S.T. M.T

ASISTEN PRATIKUM:

M. Galid



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan kehadiran Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya sehingga laporan praktikum ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban pelaksanaan praktikum struktur data dengan judul Stack. Laporan ini bertujuan untuk mendokumentasikan prosedur, hasil, serta pembahasan dari kegiatan praktikum sekaligus sebagai sarana pembelajaran bagi penulis untuk lebih memahami materi terkait.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu. Saran dan kritik yang membangun sangat diharapkan demi penyempurnaan laporan di masa mendatang. Akhir kata, penulis mengucapkan terima kasih kepada dosen pengampu dan semua pihak yang telah membantu sehingga laporan ini dapat terselesaikan.

Padang, 10 April 2026

Rayya Syaquinah Putri Hasibuan

DAFTAR ISI

Contents

KATA PENGANTAR.....	ii
DAFTAR ISI	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum.....	1
BAB II PEMBAHASAN	2
2.1 Deskripsi Materi Praktikum	2
2.2 Stack dalam JavaScript	3
2.2.1 StackArray_2511532007.....	3
2.2.3 NilaiMaksimum_2511532007.....	6
2.2.4 SiswaStack_2511532007	8
2.2.5 StackPostfix_2511532007.....	10
BAB III KESIMPULAN DAN SARAN	13
3.1 Kesimpulan	13
3.2 Saran.....	13
DAFTAR PUSTAKA	14
LAPORAN TUGAS PRAKTIKUM PEKAN 3	15

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman, pengelolaan data sangat penting agar program bisa berjalan dengan terstruktur dan efisien. Salah satu cara mengelola data adalah menggunakan struktur data, salah satunya *stack*. Stack bekerja dengan prinsip LIFO (*Last In, First Out*), yaitu data yang terakhir masuk akan menjadi data pertama yang keluar. Konsep ini sebenarnya sangat dekat dengan kehidupan sehari-hari, misalnya tumpukan buku atau piring. Praktikum ini akan membahas tentang stack, konsep LIFO. memahami stack menjadi dasar penting bagi mahasiswa dalam mempelajari struktur data dan algoritma.

1.2 Tujuan Praktikum

Tujuan dari praktikum ini adalah agar mahasiswa dapat memahami konsep dasar stack dan cara kerjanya. Selain itu, praktikum ini juga bertujuan untuk melatih mahasiswa dalam mengimplementasikan stack menggunakan bahasa pemrograman, serta memahami operasi dasar seperti push (menambah data), pop (menghapus data), dan peek (melihat data teratas). Dengan begitu, mahasiswa tidak hanya memahami teori, tetapi juga mampu menerapkannya secara langsung.

1.3 Manfaat Praktikum

Manfaat dari praktikum stack adalah membantu mahasiswa memahami cara pengolahan data secara terstruktur dan efisien. Praktikum ini juga melatih kemampuan logika dan pemecahan masalah dalam pemrograman. Selain itu, pemahaman tentang stack dapat digunakan dalam pembuatan program yang lebih kompleks,

sehingga menjadi bekal penting dalam pengembangan aplikasi di masa depan.

BAB II

PEMBAHASAN

2.1 Deskripsi Materi Praktikum

Stack merupakan salah satu struktur data yang bekerja dengan prinsip LIFO (*Last In, First Out*), yaitu data yang terakhir dimasukkan akan menjadi data pertama yang dikeluarkan. Prinsip ini membuat stack memiliki cara kerja yang terbatas namun terstruktur, karena semua operasi hanya dilakukan pada satu sisi saja, yaitu bagian paling atas yang disebut *Top of Stack*. Hal ini berbeda dengan struktur data lain seperti array atau ArrayList yang dapat diakses dari berbagai posisi. Konsep stack dapat dianalogikan dengan tumpukan benda seperti buku atau CD, di mana penambahan dan pengambilan hanya bisa dilakukan dari bagian atas. Jika ingin mengambil benda yang berada di bawah, maka benda di atasnya harus diambil terlebih dahulu.

Dalam pemrograman, stack memiliki peran yang cukup penting karena sering digunakan dalam berbagai proses, seperti *call stack* saat menjalankan fungsi dalam program, proses *backtracking* dalam pencarian solusi, evaluasi ekspresi matematika (*expression evaluation*), serta implementasi fitur “back” pada browser yang memungkinkan pengguna kembali ke halaman sebelumnya. Penggunaan stack pada kasus-kasus tersebut menunjukkan bahwa struktur ini sangat efektif untuk menangani proses yang membutuhkan urutan terbalik dari data yang masuk.

Stack memiliki tiga operasi utama, yaitu *push*, *pop*, dan *peek*. Operasi *push* digunakan untuk menambahkan data ke dalam stack, di mana data yang dimasukkan akan berada di posisi paling atas. Operasi *pop* digunakan untuk mengambil sekaligus menghapus data dari bagian atas

stack, sehingga setelah operasi ini dilakukan, posisi top akan berpindah ke data sebelumnya. Sedangkan operasi *peek* digunakan untuk melihat data yang berada di posisi paling atas tanpa menghapusnya, sehingga data tetap tersimpan dalam stack.

Selain operasi dasar, terdapat dua kondisi penting yang harus diperhatikan dalam penggunaan stack, yaitu *stack overflow* dan *stack underflow*. *Stack overflow* terjadi ketika program mencoba menambahkan data ke dalam stack yang sudah mencapai kapasitas maksimum, biasanya terjadi pada implementasi stack dengan ukuran tetap seperti array. Sementara itu, *stack underflow* terjadi ketika program mencoba mengambil atau melihat data dari stack yang kosong. Kedua kondisi ini dapat menyebabkan error jika tidak ditangani dengan baik, sehingga diperlukan pengecekan kondisi stack, seperti memastikan stack tidak kosong sebelum melakukan *pop* atau *peek*, dan memastikan stack belum penuh sebelum melakukan *push*.

Dengan demikian, stack merupakan struktur data yang sederhana namun sangat berguna dalam pemrograman karena mampu mengelola data secara teratur sesuai prinsip LIFO. Pemahaman yang baik terhadap konsep, operasi, serta penanganan kondisi khusus pada stack akan membantu dalam membangun program yang lebih efisien, terstruktur, dan minim kesalahan.

2.2 Stack dalam JavaScript

2.2.1 StackArray_2511532007

```

1 package pekan3_2511532007;
2
3 public class StackArray_2511532007 {
4     static final int MAX_2007 = 1000;
5     int top;
6     int a[] = new int [MAX_2007];
7     boolean isEmpty_2007()
8     {
9         return (top < 0);
10    }
11    StackArray_2511532007()
12    {
13        top = -1;
14    }
15    boolean push(int x)
16    {
17        if (top >= (MAX_2007 - 1)) {
18            System.out.println("Stack Overflow");
19            return false;
20        }
21        else {
22            a[++top] = x;
23            System.out.println(x + " dimasukkan dalam stack");
24            return true;
25        }
26    }

```

```

27● int pop()
28● {
29●     if (top < 0) {
30●         System.out.println("Stack Underflow");
31●     }
32●     else {
33●         int x = a[top--];
34●         return x;
35●     }
36●     return top;
37● }
38● int peek()
39● {
40●     if (top < 0) {
41●         System.out.println("Stack Underflow");
42●         return 0;
43●     }
44●     else {
45●         int x = a[top];
46●         return x;
47●     }
48● }
49● void print() {
50●     for(int i = top; i > -1; i--) {
51●         System.out.print(a[i] + " ");
52●     }
53● }
54● }

```

Langkah-langkah susunan kode program beserta fungsinya:

1. Deklarasi awal program (Package dan kelas) untuk mengelompokkan file java. 'public class stackArray_2511532007, adalah kelas utama yang berisi implementasi struktur data stack menggunakan array.
2. Mendeklarasikan variabel dan konstanta, MAX_2007 → kapasitas maksimum stack (1000 data), top → penunjuk posisi elemen teratas dalam stack, a[] → array sebagai tempat penyimpanan data stack.
3. Method isEmpty(), mengecek apakah stack kosong, jika top < 0, berarti belum ada data.
4. Inisialisasi konstruktor, top = -1 menandakan stack kosong.
5. Menambah data dengan kode program 'boolean push(int x)', Cek apakah stack penuh (top >= MAX - 1), jika penuh → tampilkan "Stack Overflow", jika tidak ++top → naikan posisi, simpan data ke array.
6. Menambahkan method pop/ menghapus data 'int pop()'. Dari kode yang tertera pada gambar, terlebih dahulu mengecek apakah stack kosong, jika kosong → "Stack Underflow", jika tidak, mengambil data teratas dan menurunkan top.
7. Method peek 'int peek()' digunakan untuk melihat elemen paling atas tanpa menghapusnya.
8. Method print 'void print()' menampilkan isi stack dari atas ke bawah dan loop dimulai dari top ke indeks 0.

Dapat disimpulkan bahwa struktur data stack bekerja dengan prinsip LIFO (*Last In, First Out*), di mana data terakhir yang

dimasukkan akan menjadi data pertama yang keluar. Implementasi stack menggunakan array memungkinkan operasi dasar seperti *push*, *pop*, dan *peek* dilakukan dengan sederhana dan terstruktur.

2.2.2 StackArrayDriver_2511532007

```

1 package pekan3_2511532007;
2
3 public class stackArrayDriver_2511532007 {
4
5     public static void main(String[] args) {
6         stackArray_2511532007 s = new stackArray_2511532007();
7         s.push(10);
8         s.push(20);
9         s.push(30);
10        System.out.println(s.pop() + "dikeluarkan dari stack ");
11        System.out.println("elemen teratas adalah : " + s.peek());
12        System.out.println("element pada stack : ");
13        s.print();
14    }
15 }
16
17 }
18

```

Langkah-langkah susunan kode program beserta fungsinya:

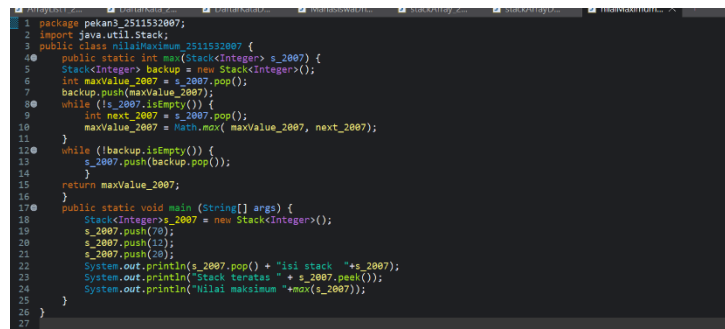
1. Deklarasi awal program (package dan class)
Digunakan untuk mengelompokkan file Java agar lebih terstruktur. `public class stackArrayDriver_2511532007` merupakan kelas utama yang berfungsi sebagai program pengujian (driver) dari implementasi stack.
2. Method main sebagai titik awal program
Method '`public static void main(String[] args)`' merupakan bagian utama yang pertama kali dijalankan saat program dieksekusi. Semua proses pengujian stack dilakukan di dalam method ini.
3. Membuat objek stack, '`stackArray_2511532007 s = new stackArray_2511532007();`' digunakan untuk membuat objek stack bernama s. Saat objek dibuat, konstruktor dijalankan sehingga nilai awal `top = -1` (stack kosong).
4. Menambahkan data kedalam stack (`push`), `s.push(10);`, `s.push(20);`, dan `s.push(30);` digunakan untuk menambahkan data ke dalam stack secara berurutan. Data akan tersusun sesuai prinsip LIFO, di mana elemen terakhir (30) berada di posisi teratas.
5. Menghapus data dari stack (`pop`), '`System.out.println(s.pop() + "dikeluarkan dari stack");`' digunakan untuk mengambil dan

menghapus elemen teratas dari stack. Data yang pertama keluar adalah 30 karena mengikuti prinsip LIFO.

6. Melihat elemen teratas (*peek*), `'System.out.println("elemen teratas adalah : " + s.peek());'` digunakan untuk menampilkan elemen teratas tanpa menghapusnya. Setelah operasi *pop*, elemen teratas menjadi 20.
7. Menampilkan seluruh isi stack(*print*), `'System.out.println("element pada stack : ");` dan `s.print();'` digunakan untuk menampilkan seluruh isi stack dari atas ke bawah. Data yang ditampilkan adalah 20 dan 10.

Program *stackArrayDriver* berfungsi sebagai penguji dari implementasi struktur data stack. Melalui program ini dapat dilihat bahwa operasi *push*, *pop*, *peek*, dan *print* berjalan sesuai dengan prinsip LIFO (*Last In, First Out*). Program ini membantu memahami bagaimana data ditambahkan, dihapus, dan ditampilkan dalam stack secara terstruktur.

2.2.3 NilaiMaksimum_2511532007



```

1 package pekan3_2511532007;
2 import java.util.Stack;
3 public class nilaiMaksimum_2511532007 {
4     public static int max(Stack<Integer> s_2007) {
5         Stack<Integer> backup = new Stack<Integer>();
6         int maxValue_2007 = s_2007.pop();
7         backup.push(maxValue_2007);
8         while (!s_2007.isEmpty()) {
9             int next_2007 = s_2007.pop();
10            maxValue_2007 = Math.max(maxValue_2007, next_2007);
11        }
12        while (!backup.isEmpty()) {
13            s_2007.push(backup.pop());
14        }
15        return maxValue_2007;
16    }
17    public static void main (String[] args) {
18        Stack<Integer> s_2007 = new Stack<Integer>();
19        s_2007.push(70);
20        s_2007.push(12);
21        s_2007.push(20);
22        System.out.println(s_2007.pop() + "isi stack " + s_2007);
23        System.out.println("Stack teratas " + s_2007.peek());
24        System.out.println("Nilai maksimum " + max(s_2007));
25    }
26 }
27

```

Langkah – langkah susunan kode program beserta fungsinya:

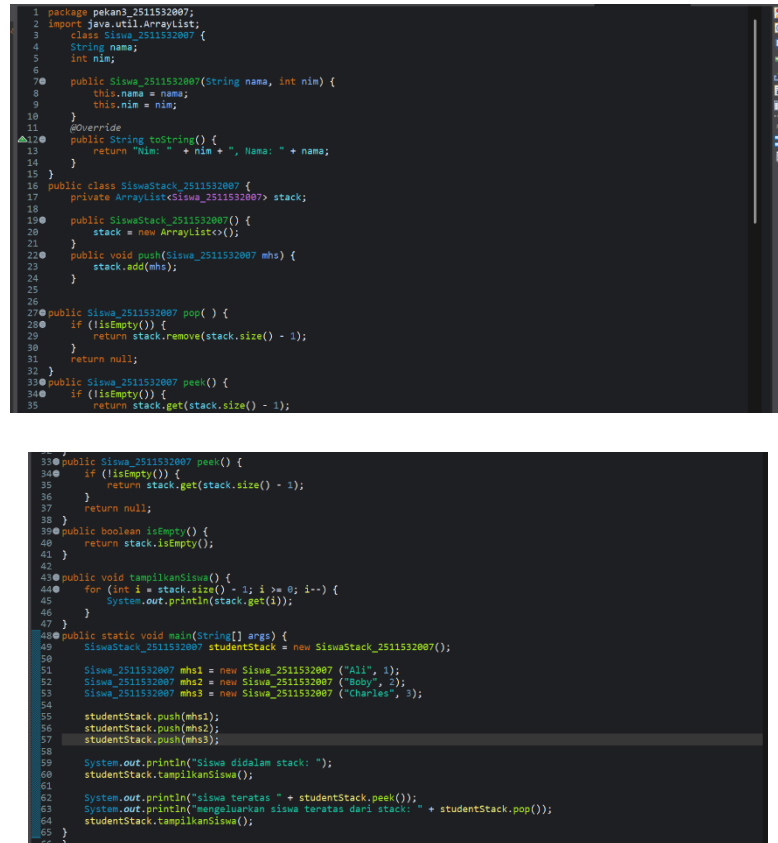
1. Deklarasi awal program (package, import, dan class) `'package pekan3_2511532007;'` digunakan untuk mengelompokkan file `'java import java.util.Stack;'` digunakan untuk memanggil library bawaan Java yaitu kelas Stack. `public class nilaiMaksimum_2511532007` merupakan kelas utama yang berisi program untuk mencari nilai maksimum pada stack.

2. Deklarasi method max `'public static int max(Stack<Integer> s_2007)'` adalah method yang digunakan untuk mencari nilai maksimum dalam stack bertipe integer. Method ini bersifat static sehingga dapat dipanggil tanpa membuat objek baru.
3. Membuat stack sementara (backup)
`'Stack<Integer> backup = new Stack<Integer>();'` digunakan sebagai tempat penyimpanan sementara agar data dari stack asli bisa dikembalikan setelah proses pencarian nilai maksimum.
4. Mengambil elemen awal sebagai nilai maksimum sementara
`'int maxValue_2007 = s_2007.pop();'` mengambil elemen teratas sebagai nilai awal maksimum `backup.push(maxValue_2007);` menyimpan nilai tersebut ke stack backup.
5. Perulangan untuk mencari nilai maksimum
`'while (!s_2007.isEmpty())'` digunakan untuk memproses semua elemen dalam stack. Setiap elemen diambil dengan `pop()`, lalu dibandingkan menggunakan `Math.max()` untuk menentukan nilai maksimum.
6. Mengembalikan isi stack ke kondisi semula `'while (!backup.isEmpty())'` digunakan untuk mengembalikan data dari stack backup ke stack utama agar data tidak hilang setelah proses pencarian maksimum.
7. Mengembalikan nilai maksimum `'return maxValue_2007;'` digunakan untuk mengembalikan hasil nilai maksimum yang ditemukan dalam stack.
8. Method main sebagai pengujian program, membuat stack: `'Stack<Integer> s_2007 = new Stack<Integer>();'`, menambahkan data: 70, 12, 20, `pop()` digunakan untuk menghapus elemen teratas, `peek()` untuk melihat elemen teratas, `max(s_2007)` untuk mencari nilai maksimum dalam stack

Program ini bertujuan untuk mencari nilai maksimum dalam struktur data stack dengan cara membandingkan setiap elemen yang ada. Penggunaan stack sementara membantu menjaga data agar tidak

hilang, meskipun pada implementasi ini masih terdapat kelemahan karena tidak semua elemen dikembalikan ke stack utama. Secara umum, program ini menunjukkan bagaimana operasi stack dan perulangan dapat digunakan untuk menyelesaikan permasalahan sederhana seperti mencari nilai maksimum.

2.2.4 SiswaStack_2511532007



```

1 package pekan3_2511532007;
2 import java.util.ArrayList;
3 class Siswa_2511532007 {
4     String nama;
5     int nim;
6
7     public Siswa_2511532007(String nama, int nim) {
8         this.nama = nama;
9         this.nim = nim;
10    }
11    @Override
12    public String toString() {
13        return "Nim: " + nim + ", Nama: " + nama;
14    }
15 }
16 public class SiswaStack_2511532007 {
17     private ArrayList<Siswa_2511532007> stack;
18
19     public SiswaStack_2511532007() {
20         stack = new ArrayList<>();
21     }
22     public void push(Siswa_2511532007 mhs) {
23         stack.add(mhs);
24     }
25
26     public Siswa_2511532007 pop() {
27         if (isEmpty()) {
28             return stack.remove(stack.size() - 1);
29         }
30         return null;
31     }
32
33     public Siswa_2511532007 peek() {
34         if (isEmpty()) {
35             return stack.get(stack.size() - 1);
36         }
37         return null;
38     }
39     public boolean isEmpty() {
40         return stack.isEmpty();
41     }
42
43     public void tampilkanSiswa() {
44         for (int i = stack.size() - 1; i >= 0; i--) {
45             System.out.println(stack.get(i));
46         }
47     }
48     public static void main(String[] args) {
49         SiswaStack_2511532007 studentStack = new SiswaStack_2511532007();
50
51         Siswa_2511532007 mhs1 = new Siswa_2511532007 ("Ali", 1);
52         Siswa_2511532007 mhs2 = new Siswa_2511532007 ("Boby", 2);
53         Siswa_2511532007 mhs3 = new Siswa_2511532007 ("Charles", 3);
54
55         studentStack.push(mhs1);
56         studentStack.push(mhs2);
57         studentStack.push(mhs3);
58
59         System.out.println("Siswa didalam stack: ");
60         studentStack.tampilkanSiswa();
61
62         System.out.println("siswa teratas " + studentStack.peek());
63         System.out.println("mengeluarkan siswa teratas dari stack: " + studentStack.pop());
64         studentStack.tampilkanSiswa();
65     }
66 }

```

Langkah – langkah susunan kode program beserta fungsinya:

1. Deklarasi awal program (package, import, dan class)
package pekan3_2511532007; digunakan untuk mengelompokkan file ,Java.import java.util.ArrayList;, digunakan untuk memanfaatkan struktur data dinamis dari Java. Terdapat dua class, yaitu Siswa_2511532007 sebagai representasi data siswa, dan SiswaStack_2511532007 sebagai implementasi stack.

2. Class Siswa (model data) Class Siswa_2511532007 memiliki atribut nama dan nim untuk menyimpan data siswa. Konstruktor digunakan untuk menginisialisasi nilai atribut saat objek dibuat. Method toString() digunakan untuk menampilkan data siswa dalam bentuk teks agar lebih mudah dibaca saat dicetak.
3. Deklarasi stack menggunakan ArrayList 'private ArrayList<Siswa_2511532007> stack;' digunakan sebagai tempat penyimpanan data stack. Berbeda dengan array biasa, ArrayList bersifat dinamis sehingga ukuran stack dapat bertambah sesuai kebutuhan.
4. Inisialisasi konstruktor Pada SiswaStack_2511532007(), stack diinisialisasi dengan 'stack = new ArrayList<>()'; sehingga siap digunakan dalam keadaan kosong.
5. Method push (menambah data) 'public void push(Siswa_2511532007 mhs)' digunakan untuk menambahkan objek siswa ke dalam stack dengan stack.add(mhs). Data yang ditambahkan akan berada di posisi paling atas (akhir ArrayList).
6. Method pop (menghapus data) 'public Siswa_2511532007 pop()' digunakan untuk menghapus dan mengembalikan elemen teratas dari stack. Jika stack tidak kosong, maka elemen terakhir dihapus menggunakan 'remove(stack.size() - 1)'. Jika kosong, method mengembalikan null.
7. Method peek (melihat data teratas) 'public Siswa_2511532007 peek()' digunakan untuk melihat elemen teratas tanpa menghapusnya. Jika stack kosong, maka akan mengembalikan null.
8. Method isEmpty (mengecek kondisi stack) 'public boolean isEmpty()' digunakan untuk mengecek apakah stack kosong dengan memanfaatkan method isEmpty() dari ArrayList.
9. Method tampilkanSiswa (menampilkan isi stack) Method ini digunakan untuk menampilkan seluruh isi stack dari atas ke

bawah menggunakan perulangan dari indeks terakhir ke indeks pertama.

10. Method main (pengujian program), membuat objek stack: studentStack, membuat objek siswa: Ali, Bobby, dan Charles, Menambahkan data ke stack menggunakan push(), Menampilkan isi stack, Menggunakan peek() untuk melihat siswa teratas, Menggunakan pop() untuk menghapus siswa teratas, Menampilkan kembali isi stack setelah penghapusan

Program ini menunjukkan implementasi struktur data stack menggunakan ArrayList untuk menyimpan objek siswa. Operasi *push*, *pop*, dan *peek* berjalan sesuai dengan prinsip LIFO (*Last In, First Out*), di mana data terakhir yang dimasukkan akan menjadi data pertama yang dikeluarkan. Penggunaan ArrayList membuat stack lebih fleksibel dibandingkan array karena ukurannya dapat berubah secara dinamis.

2.2.5 StackPostfix_2511532007

```

1 package pekan3_2511532007;
20 import java.util.Scanner;
4 public class stackPostfix_2511532007 {
50     public static int postfixEvaluate(String expression) {
6         Stack<Integer> s_2007 = new Stack<Integer>();
7         Scanner input = new Scanner(expression);
80     while (input.hasNext()) {
90         if (input.hasNextInt()) { // an operand integer
10        s_2007.push(input.nextInt());
11    } else {
12        String operator = input.next();
13        int operand2_2007 = s_2007.pop();
14        int operand1_2007 = s_2007.pop();
15        if (operator.equals("+")) {
16            s_2007.push(operand1_2007 + operand2_2007);
17        } else if (operator.equals("-")) {
18            s_2007.push(operand1_2007 - operand2_2007);
19        } else if (operator.equals("*")) {
20            s_2007.push(operand1_2007 * operand2_2007);
21        } else {
22            s_2007.push(operand1_2007 / operand2_2007);
23        }
24    }
25    input.close();
26    return s_2007.pop();
27    }
28    public static void main(String[] args) {
29        System.out.println("hasil postfix= " + postfixEvaluate("5 2 4 * + 7 -"));
30    }
31 }
32 }
33

```

Langkah – langkah susunan kode program beserta fungsinya:

1. Deklarasi awal program (package, import, dan class) package pekan3_2511532007; digunakan untuk mengelompokkan file Java. import java.util.Scanner; digunakan untuk membaca input berupa string ekspresi. import java.util.Stack; digunakan untuk memanfaatkan struktur data stack bawaan Java. public class stackPostfix_2511532007 merupakan kelas utama untuk menghitung ekspresi postfix.

2. Deklarasi method postfixEvaluate 'public static int postfixEvaluate(String expression)' merupakan method yang digunakan untuk menghitung hasil dari ekspresi postfix yang diberikan dalam bentuk string.
3. Membuat stack dan scanner 'Stack<Integer> s_2007 = new Stack<Integer>();' digunakan sebagai tempat penyimpanan operand. Scanner input = new Scanner(expression); digunakan untuk membaca setiap elemen (angka atau operator) dari string.
4. Perulangan membaca input while (input.hasNext()) digunakan untuk membaca seluruh isi ekspresi hingga selesai.
5. Memproses operand (angka) 'if (input.hasNextInt())' digunakan untuk mengecek apakah token berupa angka. Jika ya, maka angka tersebut dimasukkan ke dalam stack menggunakan push().
6. Memproses operator Jika token bukan angka, maka dianggap sebagai operator, program akan Mengambil dua operand dari stack (operand2 dan operand1), Melakukan operasi sesuai operator (+, -, *, /), Menyimpan hasilnya kembali ke dalam stack
7. Menutup scanner dan mengambil hasil akhir 'input.close();' digunakan untuk menutup scanner setelah selesai digunakan. 'return s_2007.pop();' mengembalikan hasil akhir dari perhitungan postfix.
8. Method main (pengujian program) 'System.out.println("hasil postfix= " + postfixEvaluate("5 2 4 * + 7 -"))';' digunakan untuk menguji fungsi dengan contoh ekspresi postfix.

Program ini mengimplementasikan struktur data stack untuk menyelesaikan perhitungan ekspresi postfix. Dengan memanfaatkan prinsip LIFO (*Last In, First Out*), setiap operand dan operator diproses secara berurutan hingga menghasilkan nilai

akhir. Pendekatan ini mempermudah evaluasi ekspresi matematika tanpa perlu menggunakan tanda kurung.

BAB III

KESIMPULAN DAN SARAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data stack merupakan metode penyimpanan data yang bekerja dengan prinsip LIFO (*Last In, First Out*), di mana data terakhir yang dimasukkan akan menjadi data pertama yang dikeluarkan. Implementasi stack dapat dilakukan menggunakan berbagai cara, seperti array, `ArrayList`, maupun `Stack` bawaan Java, yang masing-masing memiliki kelebihan dan keterbatasan.

Melalui beberapa program yang telah dibuat, seperti *stack array*, *stack driver*, pencarian nilai maksimum, stack objek siswa, hingga evaluasi ekspresi postfix, dapat dilihat bahwa operasi dasar stack seperti *push*, *pop*, dan *peek* memiliki peran penting dalam pengolahan data. Selain itu, praktikum ini juga menunjukkan bahwa pemahaman konsep stack tidak hanya bersifat teoritis, tetapi juga dapat diterapkan dalam berbagai kasus nyata seperti pengelolaan data, perhitungan matematika, dan simulasi proses dalam program.

Dengan demikian, pemahaman terhadap struktur data stack menjadi dasar yang penting dalam pengembangan program yang efisien, terstruktur, dan mudah dikembangkan.

3.2 Saran

Dalam pelaksanaan praktikum selanjutnya, disarankan agar mahasiswa tidak hanya memahami konsep dasar stack, tetapi juga memperhatikan penerapan logika program secara lebih mendalam, terutama dalam penanganan kondisi error seperti *stack overflow* dan *stack underflow*.

DAFTAR PUSTAKA

- [1] Telkom University Jakarta, “Stack: Konsep LIFO dan Penerapannya dalam Dunia Nyata,” 2026. [Diakses 12 April]
- [2] F. Fadhlan, “Stack dalam Javascript,” Medium, 2021. [Online]. Available: <https://medium.com/@fadhlanfm/stack-dalam-javascript-f8136744ae9> [Diakses 12 April]
- [3] GeeksforGeeks, “Implementation of Stack in JavaScript,” [Online]. Available: <https://www.geeksforgeeks.org/javascript/implementation-stack-javascript/> [Diakses 13 April]
- [4] Programiz, “JavaScript Stack Example,” [Online]. Available: <https://www.programiz.com/javascript/examples/stack> [Diakses 13 April]

LAPORAN TUGAS PRAKTIKUM PEKAN 3

1. Kelas ADT Website_2511532007

Salinan kode:

```
package pekan3_2511532007;

public class website_2511532007 {
    private String judul_2007;
    private String url_2007;

    // Constructor
    public website_2511532007(String judul, String url) {
        this.judul_2007 = judul;
        this.url_2007 = url;
    }

    // Getter
    public String getJudul() {
        return judul_2007;
    }

    public String getUrl() {
        return url_2007;
    }

    // Setter
    public void setJudul(String judul) {
        this.judul_2007 = judul;
    }

    public void setUrl(String url) {
        this.url_2007 = url;
    }

    @Override
    public String toString() {
        return "Judul: " + judul_2007 + ", URL: " + url_2007;
    }
}
```

```

1 package pekan3_2511532007;
2
3 public class website_2511532007 {
4     private String judul_2007;
5     private String url_2007;
6
7     // Constructor
8     public website_2511532007(String judul, String url) {
9         this.judul_2007 = judul;
10        this.url_2007 = url;
11    }
12
13    // Getter
14    public String getJudul() {
15        return judul_2007;
16    }
17
18    public String getUrl() {
19        return url_2007;
20    }
21
22    // Setter
23    public void setJudul(String judul) {
24        this.judul_2007 = judul;
25    }
26
27    public void setUrl(String url) {
28        this.url_2007 = url;
29    }
30
31    @Override
32    public String toString() {
33        return "Judul: " + judul_2007 + ", URL: " + url_2007;
34    }
35 }

```

Penjelasan :

Class `website_2511532007` merupakan ADT (*Abstract Data Type*) yang digunakan untuk merepresentasikan data sebuah website, yaitu berupa judul dan URL. Kedua atribut tersebut disimpan dengan akses `private` untuk menjaga keamanan data. Constructor digunakan untuk menginisialisasi nilai saat objek dibuat, sedangkan method *getter* dan *setter* berfungsi untuk mengambil dan mengubah data secara terkontrol. Method `toString()` digunakan untuk menampilkan informasi website dalam bentuk teks yang mudah dibaca. Class ini nantinya digunakan sebagai objek yang disimpan dalam struktur data stack pada program browser history.

2. Kelas Driver Browser_2511532007

Salinan kode:

```
package pekan3_2511532007;
```

```
import java.util.Scanner;
```

```
import java.util.Stack;
```

```

public class browser_2511532007 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        Stack<website_2511532007> history = new Stack<>();

        int pilihan;

        do {
            System.out.println("\n=== Browser History NIM:
2511532007 ===");
            System.out.println("1. Kunjungi Website (Push)");
            System.out.println("2. Tombol Back (Pop)");

```

```

System.out.println("3. Lihat Halaman Aktif (Peek)");
System.out.println("4. Cek Status History");
System.out.println("5. Keluar");
System.out.print("Pilihan: ");
pilihan = input.nextInt();
input.nextLine();

switch (pilihan) {
    case 1:
        System.out.print("Masukkan Judul: ");
        String judul = input.nextLine();

        System.out.print("Masukkan URL: ");
        String url = input.nextLine();

        history.push(new website_2511532007(judul, url));
        System.out.println("Berhasil mengunjungi halaman!");
        break;

    case 2:
        if (!history.isEmpty()) {
            website_2511532007 removed = history.pop();
            System.out.println("Kembali dari: " +
removed.getJudul());
        } else {
            System.out.println("History kosong!");
        }
        break;

    case 3:
        if (!history.isEmpty()) {
            System.out.println("Halaman aktif: " +
history.peek());
        } else {
            System.out.println("Tidak ada halaman yang sedang
dibuka.");
        }
        break;

    case 4:
        System.out.println("Jumlah history: " + history.size());

```

```

        System.out.println("Apakah kosong? " +
history.isEmpty());
        break;

        case 5:
            System.out.println("Program selesai.");
            break;

        default:
            System.out.println("Pilihan tidak tersedia.");
    }

    } while (pilihan != 5);

    input.close();
}
}

```

```

1 package pekan3_2511532007;
2
3 import java.util.Scanner;
4 import java.util.Stack;
5
6 public class browser_2511532007 {
7     public static void main(String[] args) {
8         Scanner input = new Scanner(System.in);
9         Stack<website_2511532007> history = new Stack<>();
10
11         int pilihan;
12
13         do {
14             System.out.println("\n===== Browser History NIM: 2511532007 =====");
15             System.out.println("1. Kunjungi Website (Push)");
16             System.out.println("2. Tombol Back (Pop)");
17             System.out.println("3. Lihat Halaman Aktif (Peek)");
18             System.out.println("4. Cek Status History");
19             System.out.println("5. Keluar");
20             System.out.print("Pilihan: ");
21             pilihan = input.nextInt();
22             input.nextLine();
23
24             switch (pilihan) {
25                 case 1:
26                     System.out.print("Masukkan Judul: ");
27                     String judul = input.nextLine();
28
29                     System.out.print("Masukkan URL: ");
30                     String url = input.nextLine();
31
32                     history.push(new website_2511532007(judul, url));
33                     System.out.println("Berhasil mengunjungi halaman!");
34                     break;
35

```

```

36         case 2:
37             if (!history.isEmpty()) {
38                 website_2511532007 removed = history.pop();
39                 System.out.println("Kembali dari: " + removed.getJudul());
40             } else {
41                 System.out.println("History kosong!");
42             }
43             break;
44
45         case 3:
46             if (!history.isEmpty()) {
47                 System.out.println("Halaman aktif: " + history.peek());
48             } else {
49                 System.out.println("Tidak ada halaman yang sedang dibuka.");
50             }
51             break;
52
53         case 4:
54             System.out.println("Jumlah history: " + history.size());
55             System.out.println("Apakah kosong? " + history.isEmpty());
56             break;
57
58         case 5:
59             System.out.println("Program selesai.");
60             break;
61
62         default:
63             System.out.println("Pilihan tidak tersedia.");
64     }
65
66     } while (pilihan != 5);
67
68     input.close();
69 }

```

Penjelasan:

Program `browser_2511532007` merupakan kelas driver yang digunakan untuk mensimulasikan browser history dengan memanfaatkan struktur data Stack. Program ini menyediakan menu interaktif untuk melakukan beberapa operasi, yaitu mengunjungi website (*push*), kembali ke halaman sebelumnya (*pop*), melihat halaman yang sedang aktif (*peek*), serta mengecek jumlah dan kondisi stack (*size* dan *isEmpty*).

Saat pengguna memilih kunjungi website, data judul dan URL akan dimasukkan ke dalam stack. Jika memilih tombol back, maka halaman terakhir akan dihapus sesuai prinsip LIFO (*Last In, First Out*). Fitur peek digunakan untuk melihat halaman yang sedang aktif tanpa menghapusnya, sedangkan fitur cek status menampilkan jumlah history yang tersimpan.

Program ini juga sudah menangani kondisi stack kosong sehingga tidak terjadi error saat melakukan operasi pop atau peek.

3. Output program

```

37     if (!history.isEmpty()) {
38         website 2511532007 removed = his
39         System.out.println("Kembali dari
40     } else {
41         System.out.println("History koso
42     }
43     break;
44
45     case 3:
46         if (!history.isEmpty()) {
47             System.out.println("Halaman akti
48         } else {
49             System.out.println("Tidak ada ha
50         }
51         break;
52
53     case 4:
54         System.out.println("Jumlah history:
55         System.out.println("Apakah kosong? "
56         break;
57
58     case 5:
59         System.out.println("Program selesai.
60         break;
61
62     default:
63         System.out.println("Pilihan tidak te
64     }
65     } while (pilihan != 5);
66     input.close();
67 }
68 }
69 }
70 }
71 }

```

```

===== Browser History NIM: 2511532007 =====
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 1
Masukkan Judul: SSO UNAND
Masukkan URL: https://sso.unand.ac.id/login
Berhasil mengunjungi halaman!

===== Browser History NIM: 2511532007 =====
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 2
Kembali dari: SSO UNAND

===== Browser History NIM: 2511532007 =====
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 3
History kosong!

===== Browser History NIM: 2511532007 =====
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 5
Program selesai.

```

Penjelasan:

Output program menunjukkan cara kerja struktur data stack dalam mensimulasikan browser history. Saat pengguna memilih menu kunjungi website (push), data berupa judul dan URL dimasukkan ke dalam stack dan ditampilkan pesan bahwa halaman berhasil dikunjungi. Ketika memilih tombol back (pop), data terakhir yang dimasukkan akan dihapus dan ditampilkan sebagai halaman yang ditinggalkan, sesuai dengan prinsip LIFO (Last In, First Out). Jika pengguna mencoba melakukan pop saat stack kosong, program akan menampilkan pesan "History kosong!" tanpa error, yang menunjukkan bahwa kondisi underflow sudah ditangani dengan baik. Program akan terus menampilkan menu karena menggunakan perulangan hingga pengguna memilih keluar, dan ketika memilih opsi keluar, program akan berhenti dengan pesan "Program selesai."